
AN0041-EC NB-IoT SoftSIM API 接口说明_V1.1

文档说明

本文档用于帮助用户如何使用移芯平台进行 softSIM 相关功能的开发，对相关 API 接口进行说明。

目录

1 引言	3
1.1 文档目的	3
2 API 接口	3
2.1 SIM API	3
2.2 FLASH API	4
2.2.1 FLASH 读取	5
2.2.2 FLASH 写	6
2.2.3 FLASH 擦除	6
2.3 安全 API	6
2.3.1 注册 ECDA 方法	6
2.3.2 随机数 API	7
2.3.3 AES 加解密 API	7
2.3.4 SHA 256 散列函数	7
2.3.5 ECDH 曲线注册函数	8
2.3.6 ECDH 产生公私钥	8
2.3.7 判断公钥是否属于曲线	8
2.3.8 计算共享秘钥 SAB	9
2.3.9 ECDSA 数字签名	9
2.3.10 ECDSA 签名验证	9
2.3.11 产生 CSR 签名	10
2.3.12 产生 CMAC	10
2.4 LOG API	10
2.4.1 Unilog 通用打印接口	10
2.4.2 Unilog 普通 dump 数据接口	11
2.4.3 Unilog 字符串打印接口	11
3 AT 命令接口	12
4 版本	12
5 关于我们	13

1 引言

1.1 文档目的

本文详细介绍 SoftSIM 的 SDK API, 以便相关开发人员使用和学习。

2 API 接口

2.1 SIM 接口

2.1.1 SIM RESET

- 头文件

```
#include "ecSoftsimApt.h"
```

- 原型

```
void SoftSimReset(UINT16 *atrLen, UINT8 *atrData);
```

参数

* Description: softsim reset callback function

* param[in] null

* param[out] unsigned short *atrLen, the pointer to the length of ATR

* param[out] unsigned char *atrData, the pointer to the ATR data. 返回值

说明:

SIM reset 函数采用信号量机制进行消息同步, 所以目前我们在该接口的消息处理的时候有特定的说明, 举例如下:

```
//创建 semaphore 信号
osSemaphoreId_t sem = osSemaphoreNew(1U, 0, PNULL);
osStatus_t osState = osOK;
//卡商在此添加 reset 消息封装和发送
/*
 * create signal/msg
 * send signal/msg to softsim task
 * release sem after process done on softsim task
 */
//softsim vender add:
/*
 * wait for sem
if ((osState = osSemaphoreAcquire(sem, portMAX_DELAY)) != osOK)
{
    OsaDebugBegin(FALSE, osState, 0, 0);
    OsaDebugEnd();
}
```

```

    }

```

2.1.1.2 SIM APDU

- 头文件

```
#include "ecSoftsimApt.h"
```

- 原型

```
void SoftSimAptReq(UINT16 txDataLen, UINT8 *txData, UINT16 *rxDataLen, UINT8 *rxData)
```

参数

* Description: soft sim APDU request/response callback function

* param[in] unsigned short txDataLen, the length of tx data

* param[in] unsigned char *txData, the pointer to the tx data.

* param[out] unsigned short *rxDataLen, the pointer to the length of rx data

* param[out] unsigned char *rxData, the pointer to the rx data. 说明:

说明:

该函数和 SIM RESET 一样，主体都是由卡商实现，该 APDU 处理采用 semaphore 机制，所以需要卡商实现 APDU 消息的封装和发送，例子如下：

```

osSemaphoreId_t sem = osSemaphoreNew(1U, 0, PNULL);

osStatus_t osState = osOK;

/*
 * create signal/msg
 * send signal/msg to softsim task
 * release sem after process done on softsim task
 */
//softsim vender add:

/*
 * wait for sem
 */
if((osState = osSemaphoreAcquire(sem, portMAX_DELAY)) != osOK)
{
    OsaDebugBegin(FALSE, osState, 0, 0);
    OsaDebugEnd();
}

```

2.2 FLASH API

当前 FLASH 共有 32KB，共划分为 8 个扇区，即每个扇区大小 4KB。

```

/*-----FLASH MAPPING-----/
*
* ----- ecSoftSimHeaderSectorAddr: 0x344000

```

```

*      |      |
*      |      HEADER (4k)      |
*      |----- SOFTSIM_FLASH_BASE_ADDRESS: 0x344000 + 4k
*      |      |
*      |      DATA1 (4k)      |
*      |----- SOFTSIM_FLASH_DATA_ADDRESS: 0x344000 + 8k
*      |      |
*      |      DATA2 (4k)      |
*      |----- SOFTSIM_FLASH_SWAP_ADDRESS : 0x344000 + 12k
*      |      |
*      |      DATA3(4K)      |
*      |----- SOFTSIM_FLASH_FDT_ADDRESS : 0x344000 + 16K
*      |      |
*      |      DATA4 (4k)      |
*      |-----
*      |      |
*      |      未用      |
*      |-----
*      |-----
*      |      |
*      |      未用      |
*      |-----
*      |-----
*      |      |
*      |      未用      |
*      |-----
*-----FLASH MAPPING END -----*/

```

2.2.1 FLASH 读取

- 头文件

```
#include "ecSoftsimApt.h"
```

- 原型

```
INT8 SoftsimReadRawFlash(UINT32 addr, UINT8 *pBuffer, UINT32 bufferSize)
```

参数

* input:

addr: 要读取的 flash 地址

bufferSize 要读取的 flash 大小

* output:

pBuffer : 保存 data buffer

- 返回值

成功, 返回 0; 失败, 返回-1;

2.2.2 FLASH 写

- 头文件

```
#include "ecSoftsimApt.h"
```

- 原型

INT8 SoftsimWriteToRawFlash(UINT32 addr, UINT8 *pBuffer, UINT32 bufferSize)

参数

* input:

addr: 要写入的 flash 地址

pBuffer 要写入的 data

bufferSize 要写入的 data 大小

* output:

N/A

- 返回值

成功, 返回 0; 失败, 返回-1;

2.2.3 FLASH 擦除

- 头文件

```
#include "ecSoftsimApt.h"
```

- 原型

INT8 SoftsimEraseRawFlash (UINT32 sectorAddr, UINT32 size)

参数

* input:

sectorAddr: 要擦除的 flash 地址

size 要擦除的 flash 大小

* output:

N/A

- 返回值

成功, 返回 0; 失败, 返回-1;

注意: flash 的特性要求, 擦除都是以 4K 为一个扇区进行擦除, 所以该函数会对 sectorAddr 的地址进行判断, 如果是从 0x344000 地址开始的并且是 4K 的倍数, 擦除才可以执行。

2.3 安全 API

2.3.1 注册 ECDA 方法

- 头文件

```
#include "ecsoftsiminterface.h"
```

- 原型

UINT8 ecSoftSimMethodRegister (void)

说明：在调用安全 API 之前，需要先调用该函数注册 ECP/ECDSA 相关方法和初始化安全 context
在 softSIM task 调用一次即可

- 参数

- * input: N/A

- * output: N/A

- 返回值

成功，返回 0； 失败，返回-1；

2.3.2 随机数 API

- 头文件

- ```
#include "ecsoftsiminterface.h"
```

- 原型

- ```
UINT8 ecSoftSimRandom (UINT16 length, UINT8 *random)
```

- 参数

- * input: length: the defined length of required random

- * output: random datas

- 返回值

成功，返回 0； 失败，返回-1；

2.3.3 AES 加解密 API

- 头文件

- ```
#include "ecsoftsiminterface.h"
```

- 原型

- ```
UINT8 ecSoftSimAes(UINT8 *key_in, UINT8 *data_in, UINT32 length, UINT8 *iv_in, UINT8 *data_out,  
UINT8 mode, UINT8 encryption)
```

- 参数

- key_in [input]: the 128bits AES key

- data_in [input]: the original data

- length [input]: the original data length

- iv_in [input]: the CBC encryption IV value

- data_out [output]: the output data

- mode [input]: AES mode 1. CBC mode;0:ECB mode

- encryption [input]: 1: encryption ;0: decryption

- 返回值

成功，返回 0； 失败，返回-1；

2.3.4 SHA 256 散列函数

- 头文件

- ```
#include "ecsoftsiminterface.h"
```

- 原型

- ```
UINT8 ecSoftSimSha256(const UINT8 *content, UINT16 content_len, UINT8 md[32])
```

- 参数

- * input:

content : the original data
content_len : the original data len

* output:

md: 32 Bytes md value

- 返回值
成功, 返回 0; 失败, 返回-1;

2.3.5 ECDH 曲线注册函数

- 头文件

```
#include "ecsoftsiminterface.h"
```

- 原型

```
UINT8 ecSoftSimRegisterCurve(const UINT8 *ec_name)
```

- 参数

* input:

ec_name : the input curve name, example: brainpoolP256r1

* output:

- 返回值
成功, 返回 0; 失败, 返回-1

2.3.6 ECDH 产生公私钥

- 头文件

```
#include "ecsoftsiminterface.h"
```

- 原型

```
UINT8 ecSoftSimCreateEcpKeyPair (UINT8 sk[32], UINT8 pk[64])
```

- 参数

* output:

sk : the 32 BYTE private key

pk : the 64 BYTE public key

* input:

- 返回值
成功, 返回 0; 失败, 返回-1;

2.3.7 判断公钥是否属于曲线

- 头文件

```
#include "ecsoftsiminterface.h"
```

- 原型

```
UINT8 ecSoftSimCheckPkWhetherOnCurve(UINT8 pk[64])
```

- 参数

* input:

pk : the public key of ECDH curve

*output:

- 返回值
成功, 返回 0; 失败, 返回-1;

2.3.8 计算共享密钥 SAB

- 头文件

```
#include "ecsoftsiminterface.h"
```

- 原型

```
UINT8 ecSoftSimCreateEcdhKey (UINT8 sk[32], UINT8 pk[64], UINT8 sab[64])
```

- 参数

* input:

sk: private key

pk: public key

* output:

sab : the share key

- 返回值

成功, 返回 0; 失败, 返回-1;

2.3.9 ECDSA 数字签名

- 头文件

```
#include "ecsoftsiminterface.h"
```

- 原型

```
UINT8 ecSoftSimEcdsaSign(UINT8 *dgst, UINT16 dgstLen, UINT8 sk[32], UINT8 sign[64])
```

- 参数

* input:

dgst : the original data need to be signed

dgstLen : the original data length

sk: : the private key

* output:

sign : the sign data

- 返回值

成功, 返回 0; 失败, 返回-1;

2.3.10 ECDSA 签名验证

- 头文件

```
#include "ecsoftsiminterface.h"
```

- 原型

```
UINT8 ecSoftSimEcdsaSignVerify (UINT8 *dgst, UINT16 dgst_len, UINT8 sign[64],UINT8 pk[64])
```

- 参数

* input:

dgst : the original data

dgst_len : the original data length

sign : the signature data

pk: : the peer public key

*output:

- 返回值

成功，返回 0； 失败，返回-1；

2.3.11 产生 CSR 签名

- 头文件

```
#include "ecsoftsiminterface.h"
```

- 原型

```
UINT8 ecSoftSimCreateCsr(UINT8 sk[32], UINT8 pk[64], UINT8 *subject,UINT8 *outbuf, UINT16 *iolen)
```

- 参数

* input:

```
sk    : the private key
pk    : the public key
subject : the CSR subject
```

* output:

```
outbuf: the CSR data
iolen  : the CSR len
```

- 返回值

成功，返回 0； 失败，返回-1；

2.3.12 产生 CMAC

- 头文件

```
#include "ecsoftsiminterface.h"
```

- 原型

```
UINT8 ecSoftSimCreateCmac(UINT8 *Key, UINT8 *Message, UINT32 MessLen, UINT8 *Cmac)
```

- 参数

* input:

```
key : the input key
Message : the data
MessLen : the input data len
```

* output:

```
Cmac: the CMAC data
```

- 返回值

成功，返回 0； 失败，返回-1；

2.4 LOG API

SDK 支持 printf, Segger_RTT 打印 LOG 的方式，但是不建议用户使用，因为它们打印效率低，在 NB 通讯过程中会严重影响调度的实时性。推荐使用 EC 的高速 LOG 接口 UNILog，该接口通过 SOC 内部硬件加速实现低延迟高效率的 LOG 打印，该接口通过高速串口将 LOG 发送到 PC 上，用户可以通过专用的 LOG 查看工具 EPAT 离线或者实时查看 LOG。本章节主要介绍如何使用 UNILog API 接口。

2.4.1 Unilog 通用打印接口

- 头文件

```
#include " debug_trace.h"
```

- 原型

ECOMM_TRACE(moduleID, subID, debugLevel, argLen, format, ...)

- 参数

moduleID: LOG 模块 ID 信息, **SoftSIM 模块统一采用 ID: UNILog_SoftSIM**

subID: 相同 moduleID 下的 subID 信息,用于区分相同模块下不同的函数内的打印

debugLevel: 打印级别信息

argLen: 打印参数的个数

format: 打印的格式化输出内容

... : 不定长打印输入参数

注: 目前支持的格式为: %d, %x, %f, %u。

- 返回值

N/A

2.4.2 Unilog 普通 dump 数据接口

- 头文件

```
#include "debug_trace.h"
```

- 原型

ECOMM_DUMP(moduleID, subID, debugLevel, format, dumpLen, dump)

- 参数

moduleID: LOG 模块 ID 信息, **SoftSIM 模块统一采用 ID: UNILog_SoftSIM**

subID: 相同 moduleID 下的 subID 信息,用于区分相同模块下不同的函数内的打印

debugLevel: 打印级别信息

format: 打印的格式化输出内容

dumpLen: dump 数据的长度

dump: dump 数据的内容

- 返回值

N/A

注:不建议用户打印太长的数据,如超过 200byte,大概率会出现 log 丢失。

2.4.3 Unilog 字符串打印接口

- 头文件

```
#include "debug_trace.h"
```

- 原型

ECOMM_STRING(moduleID, subID, debugLevel, format, string)

- 参数

moduleID: LOG 模块 ID 信息, **SoftSIM 模块统一采用 ID: UNILog_SoftSIM**

subID: 相同 moduleID 下的 subID 信息,用于区分相同模块下不同的函数内的打印

debugLevel: 打印级别信息

format: 打印的格式化输出内容

string: 待打印的字符串信息

3 AT 命令接口

3.1 使能 SOFTSIM 开关

AT+ESIMSWITCH = value

value 参数说明:

- 1: 打开 SOFTSIM 开关
- 0: 关闭 SOFTSIM 开关, 默认即为关闭

3.2 查询当前 SOFTSIM 功能

AT+ESIMSWITCH?

返回值:

- 1: 当前已经打开 SOFTSIM
- 0: 当前没有打开 SOFTSIM

4 版本

版本	日期	Comment
V1.0	2020-10-15	Created
V1.1	2020-12-15	Modified

5 关于我们

上海移芯通信科技有限公司坐落于上海张江，公司于 2017 年 2 月成立，从事蜂窝移动通信芯片的研发和销售。公司产品主要应用于广域物联网通信领域，致力于设计最具性价比的蜂窝物联网芯片。公司团队在蜂窝通信芯片上有着辉煌历史和丰富经验。公司所有核心技术全部自研，包括算法&架构、射频、基带、SoC、协议栈软件、平台&应用软件和硬件方案等。

移芯通信首款自主研发的超低功耗 NB-IoT 芯片 EC616 于 2019 年中量产，被绝大部分头部模组企业采用，现已大批量应用于全国各区域各行业。下一代超高集成度 NB-IoT 芯片 EC616S 于 20 年底量产，超低功耗 4G Cat.1 芯片 EC618 工程样片阶段，5G 芯片正在研发中。

上海移芯通信科技有限公司

地址：中国上海市浦东新区祥科路 298 号佑越国际 1 幢 7 层

邮编：201210

电话：

电邮：

网址：<http://www.eigencomm.com>

技术支持窗口

电邮：support@eigencomm.com